

RV College of Engineering[®], Bengaluru-560 059
Autonomous Institution affiliated to Visvesvaraya Technological University
Model Question Paper

COURSE CODE: AI253IA	SEM: V
COURSE TITLE: ARTIFICIAL NEURAL NETWORKS AND DEEP LEARNING	
Duration of Paper: 03 Hrs.	

Instructions to Candidates:

1. Answer all questions from Part A
2. Any 5 Full questions from Part B choosing one from each side. (Question No.2 is compulsory)

Question No	PART A	Marks	BTL	CO
1.1	Consider $[x_1, x_2, x_3] = [0.3, 0.5, 0.6]$ and weight $[w_1, w_2, w_3] = [0.2, 0.1, -0.3]$. What is the net input of the neural network?	02	2	1
1.2	Give the difference between synaptic link and activation link in a signal flow diagram?	02	1	1
1.3	Diagrammatically represent the general architecture of a multi-layer perceptron.	02	2	1
1.4	List any two stopping criteria for backpropagation rule?	02	1	1
1.5	Mathematically define the convolutional operations from the q th layer to the $(q + 1)$ layer	02	2	1
1.6	Give the difference between real data augmentation and synthetic data augmentation	02	1	1
1.7	With a neat diagram, define any two activation functions	02	1	1
1.8	Justify why bidirectional RNN is used for secondary protein structure prediction	02	4	1
1.9	Give any two architectural differences between AlexNet and VGG Net	02	2	2
1.10	What is the main goal of monte carlo tree search? Give an example	02	1	1
	PART B			
2(a)	With a neat diagram, explain the working of multilayer feed forward network	6	2	2
2(b)	Consider the given scenario: The vectors $(1, 1, 1, 1)$ and $(-1, 1, -1, -1)$ belong to class one (target 1) and vectors $(1, 1, 1, -1)$ and $(1, -1, -1, 1)$ belong to class -1. learning rate as 1 and initial weights as 0, Threshold = 0.2. (i) Draw the network architecture (ii) Write the activation function (iii) Perform the classification using perceptron network for 3 epochs	10	1	2
3(a)	Briefly discuss the architecture of CNN with a neat diagram	6	3	2
3(b)	Considering the given matrix, perform the different convolutional operations considering zero padding and stride as	10	1	3

	2.																																																
	Input 6x6 matrix <table><tr><td>3</td><td>0</td><td>1</td><td>2</td><td>7</td><td>4</td></tr><tr><td>1</td><td>5</td><td>8</td><td>9</td><td>3</td><td>1</td></tr><tr><td>2</td><td>7</td><td>2</td><td>5</td><td>1</td><td>3</td></tr><tr><td>0</td><td>1</td><td>3</td><td>1</td><td>7</td><td>8</td></tr><tr><td>4</td><td>2</td><td>1</td><td>6</td><td>2</td><td>8</td></tr><tr><td>2</td><td>4</td><td>5</td><td>2</td><td>3</td><td>9</td></tr></table> + <table><tr><td>1</td><td>0</td><td>-1</td></tr><tr><td>1</td><td>0</td><td>-1</td></tr><tr><td>1</td><td>0</td><td>-1</td></tr></table>	3	0	1	2	7	4	1	5	8	9	3	1	2	7	2	5	1	3	0	1	3	1	7	8	4	2	1	6	2	8	2	4	5	2	3	9	1	0	-1	1	0	-1	1	0	-1			
3	0	1	2	7	4																																												
1	5	8	9	3	1																																												
2	7	2	5	1	3																																												
0	1	3	1	7	8																																												
4	2	1	6	2	8																																												
2	4	5	2	3	9																																												
1	0	-1																																															
1	0	-1																																															
1	0	-1																																															
	OR																																																
4(a)	Elaborate how pooling can be used to detect the edges in an image. For the given matrix in Q3b, apply max and average pooling considering the size of 3x3.	06	3	2																																													
4(b)	Assume a series of data stored as x-ray images with reference to classifying the image as presence or absence of chest congestion. The number of images are less for training, Identify and justify the technique use to increase the training set and explain the process in detail.	10	1	3																																													
5(a)	Illustrate the working of a language model with a neat sketch.	08	2	2																																													
5(b)	Summarize the difference between Backpropagation Through Time and Bidirectional Recurrent Networks.	08	2	3																																													
	OR																																																
6(a)	Illustrate the working of LSTM with appropriate diagram and example	08	2	2																																													
6(b)	Justify RNN is more suitable to implement temporal recommender systems when compared to other NN.	08	2	3																																													
7(a)	Briefly discuss the various challenges faced in reinforcement learning	10	3	2																																													
7(b)	With respect to Atari Setting, explain deep learning as a function approximator.	6	2	4																																													
	OR																																																
8(a)	Briefly discuss the Modeling States Versus State-Action Pairs	10	3	2																																													
8(b)	With an appropriate example, explain the straw man algorithm.	06	2	4																																													
9(a)	Explain the concept of Attention Mechanisms in neural networks. How do Attention Mechanisms enhance performance in tasks such as Machine Translation?	08	2	2																																													
9(b)	Describe the architecture and functioning of a Neural Turing Machine	08	3	3																																													
	OR																																																
10(a)	Discuss the process of training a Generative Adversarial Network (GAN). What are the challenges associated with training GANs, and how can techniques such as Wasserstein GANs help improve training stability?	8	2	2																																													
10(b)	Compare Generative Adversarial Networks (GANs) with Variational Autoencoders (VAEs) in terms of architecture, training process, and applications	8	3	3																																													

Artificial Neural Networks and Deep Learning

Part A: Solved Questions (Marks 02)

1.1 Consider $[x_1, x_2, x_3] = [0.3, 0.5, 0.6]$ and weight $[w_1, w_2, w_3] = [0.2, 0.1, -0.3]$. What is the net input of the neural network?

The net input (v) is calculated as the dot product of the input vector and the weight vector (sum of products):

$$v = \sum_{i=1}^3 x_i w_i$$

Calculation:

$$v = (0.3 \times 0.2) + (0.5 \times 0.1) + (0.6 \times -0.3)$$

$$v = 0.06 + 0.05 - 0.18$$

$$v = 0.11 - 0.18$$

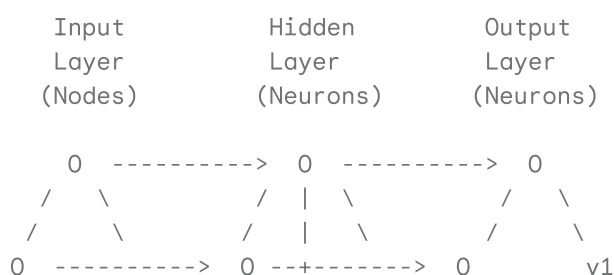
$$\mathbf{v} = -0.07$$

1.2 Give the difference between synaptic link and activation link in a signal flow diagram.

- **Synaptic Link:** Represents the linear dependence of a signal. It connects an input node j to a neuron k and carries a weight w_{kj} . It multiplies the input signal x_j by the weight w_{kj} (i.e., $y = w_{kj}x_j$).
- **Activation Link:** Represents the non-linear mapping of the signal. It typically connects the output of the linear combiner (summing junction) to the final output node of the neuron. It applies the activation function $\varphi(\cdot)$ to the net input (i.e., $y = \varphi(v)$).

1.3 Diagrammatically represent the general architecture of a Multilayer Perceptron (MLP).

An MLP consists of an Input Layer, one or more Hidden Layers, and an Output Layer.





Arrows represent weighted synaptic connections.

1.4 List any two stopping criteria for the Backpropagation rule.

1. **Gradient Magnitude:** The Euclidean norm of the gradient vector falls below a sufficiently small threshold (indicating the network has reached a minimum).
2. **Epoch Limit:** The number of epochs (iterations) reaches a predefined maximum value.
3. (Alternative) **Generalization Loss:** The error on a validation set starts to increase (Early Stopping).

1.5 Mathematically define the convolutional operations from the q^{th} layer to the $(q + 1)^{th}$ layer.

For a 2D convolution, the value of a unit x_{ij}^{q+1} in the feature map of layer $q + 1$ is given by:

$$x_{ij}^{q+1} = \sigma \left(b + \sum_r \sum_c w_{rc} \cdot x_{(i+r)(j+c)}^q \right)$$

Where:

- σ is the activation function.
- b is the bias.
- w_{rc} represents the weights of the kernel (filter) at row r and column c .
- x^q is the input from the previous layer at the corresponding spatial position.

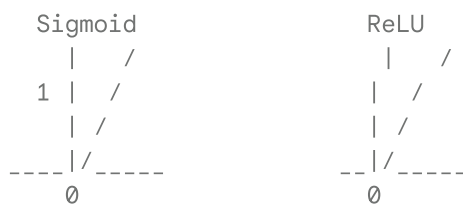
1.6 Give the difference between real data augmentation and synthetic data augmentation.

- **Real Data Augmentation:** Involves transforming the *existing* real training data to create variations. Examples include rotating, flipping, scaling, cropping, or adding noise to real images.
- **Synthetic Data Augmentation:** Involves generating completely *new* artificial data points that resemble the real distribution, often using Generative Adversarial Networks (GANs) or simulation engines, rather than just modifying existing samples.

1.7 With a neat diagram, define any two activation functions.

1. **Sigmoid Function:** Squashes input into a range $(0, 1)$. Used often in output layers for probability.
 - Equation: $\varphi(v) = \frac{1}{1+e^{-v}}$
 - Shape: S-shaped curve.
2. **ReLU (Rectified Linear Unit):** Outputs the input directly if positive, else zero.

- Equation: $\varphi(v) = \max(0, v)$
- Shape: Flat line at 0 for $x < 0$, linear slope for $x > 0$.



1.8 Justify why Bidirectional RNN is used for secondary protein structure prediction.

Protein secondary structure at a specific location depends not only on the amino acids preceding it (upstream) but also on those following it (downstream).

- A standard RNN only captures past context (N-terminal to C-terminal).
- A **Bidirectional RNN (BiRNN)** processes the sequence in both directions (forward and backward), allowing the network to use the complete context of the entire amino acid sequence to predict the structure at any given point.

1.9 Give any two architectural differences between AlexNet and VGG Net.

1. **Filter Size:** AlexNet uses large receptive fields (filters) in the first layers (e.g., 11×11 and 5×5). VGG Net uses very small, fixed receptive fields (3×3) throughout the entire network.
2. **Depth:** VGG Net is significantly deeper (typically 16 or 19 weight layers) compared to AlexNet (8 weight layers).

1.10 What is the main goal of Monte Carlo Tree Search (MCTS)? Give an example.

- **Goal:** To determine the most promising move in a decision process by balancing **exploration** (looking at unvisited states) and **exploitation** (visiting states known to differ high rewards). It builds a search tree incrementally to approximate the theoretic value of moves.
- **Example: AlphaGo** used MCTS to play the game of Go. It simulated thousands of random games from the current board position to estimate the probability of winning for each possible move.

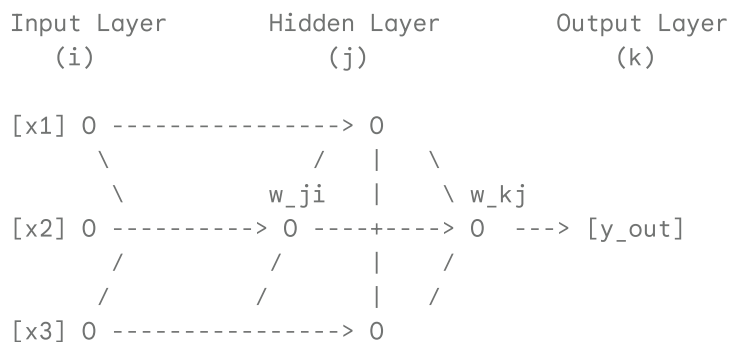
Part B: Solved Questions

2(a) With a neat diagram, explain the working of a multilayer feed-forward network. (6 Marks)

A Multilayer Feed-Forward Network (MLFFN) is a type of neural network where connections between nodes do not form a cycle. It consists of an input layer, one or more hidden layers, and an output layer.

Working Principle:

1. **Input Phase:** The input layer receives the initial data signals $(x_1, x_2, \dots, x_n)$. No computation happens here; it simply passes the values to the first hidden layer.
2. **Forward Propagation (Hidden Layers):**
 - Each neuron in the hidden layer receives signals from the previous layer.
 - It computes the **Weighted Sum (Net Input)**: $v_j = \sum (w_{ij} \cdot x_i) + b_j$.
 - It applies an **Activation Function** (e.g., Sigmoid, ReLU) to this sum: $y_j = \varphi(v_j)$.
 - This output becomes the input for the next layer.
3. **Output Phase:** The final hidden layer passes signals to the output layer neurons, which perform similar computations to produce the final prediction or classification.
4. **Learning (Training):** The network compares the output with the target value to calculate error. Using algorithms like Backpropagation, it updates the weights to minimize this error over time.

Diagram:

- **Feed-forward:** Information flows only in one direction, from input to output ($i \rightarrow j \rightarrow k$).
- **Weights (w):** Determine the strength of the connection.
- **Biases (b):** Allow shifting the activation function.

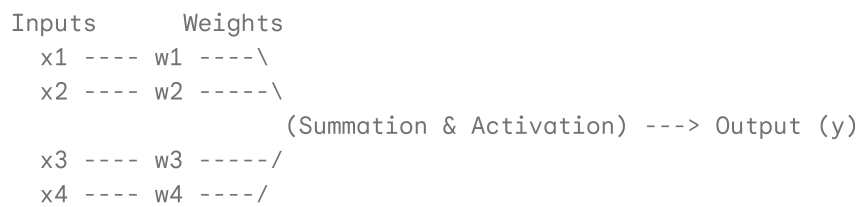
2(b) Perceptron Classification Problem (10 Marks)**Given Scenario:**

- **Class 1 (Target $t = 1$):**
 - $S_1: (1, 1, 1, 1)$
 - $S_2: (-1, 1, -1, -1)$
- **Class -1 (Target $t = -1$):**
 - $S_3: (1, 1, 1, -1)$
 - $S_4: (1, -1, -1, 1)$
- **Parameters:**
 - Initial Weights $W_0 = [0, 0, 0, 0]$

- Learning Rate $\alpha = 1$
- Threshold $\theta = 0.2$

(i) Network Architecture

A simple single-layer perceptron with 4 inputs and 1 output.



(ii) Activation Function

Since the threshold is $\theta = 0.2$ and targets are bipolar (1, -1):

$$y = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > 0.2 \\ -1 & \text{if } y_{in} \leq 0.2 \end{cases}$$

(iii) Classification for 3 Epochs

Weight Update Rule: $W_{new} = W_{old} + \alpha(t - y)x$ (Or simplified perceptron rule: If error, $W_{new} = W_{old} + \alpha tx$). We will use the simplified rule which effectively adds the input vector to weights if target is 1 and subtracts if target is -1.

Epoch 1

Step	Input Vector (x)	Target (t)	Net Input $y_{in} = \sum w_i x_i$	Output y (Threshold 0.2)	Error?	Weight Update ($W_{new} = W_{old} + tx$)
1	(1, 1, 1, 1)	1	0	-1	Yes	$[0, 0, 0, 0] + (1)[1, 1, 1,$
2	(-1, 1, -1, -1)	1	$-1 + 1 - 1 - 1 = -2$	-1	Yes	$[1, 1, 1, 1] + (1)[-1, 1,$
3	(1, 1, 1, -1)	-1	$0 + 2 + 0 + 0 = 2$	1	Yes	$[0, 2, 0, 0] - (1)[1, 1, 1,$
4	(1, -1, -1, 1)	-1	$-1 - 1 + 1 + 1 = 0$	-1	No	No Change

Epoch 2 (Starting Weights: [-1, 1, -1, 1])

Step	Input Vector (x)	Target (t)	Net Input $y_{in} = \sum w_i x_i$	Output y	Error?	Weight Update
1	(1, 1, 1, 1)	1	$-1 + 1 - 1 + 1 = 0$	-1	Yes	$[-1, 1, -1, 1] + [1, 1, 1, 1]$

2	$(-1, 1, -1, -1)$	1	$0 + 2 + 0 - 2 = 0$	-1	Yes	$[0, 2, 0, 2] + [-1, 1, -1, -1]$
3	$(1, 1, 1, -1)$	-1	$-1 + 3 - 1 - 1 = 0$	-1	No	No Change (Since $0 \leq 0.2$)
4	$(1, -1, -1, 1)$	-1	$-1 - 3 + 1 + 1 = -2$	-1	No	No Change

Epoch 3 (Starting Weights: $[-1, 3, -1, 1]$)

Step	Input Vector (x)	Target (t)	Net Input $y_{in} = \sum w_i x_i$	Output y	Error?	Weight Update	Final Weights (W)
1	$(1, 1, 1, 1)$	1	$-1 + 3 - 1 + 1 = 2$	1	No	No Change ($2 > 0.2$)	$[-1, 3, -1, 1]$
2	$(-1, 1, -1, -1)$	1	$1 + 3 + 1 - 1 = 4$	1	No	No Change	$[-1, 3, -1, 1]$
3	$(1, 1, 1, -1)$	-1	$-1 + 3 - 1 - 1 = 0$	-1	No	No Change	$[-1, 3, -1, 1]$
4	$(1, -1, -1, 1)$	-1	$-1 - 3 + 1 + 1 = -2$	-1	No	No Change	$[-1, 3, -1, 1]$

Conclusion: The network has converged after Epoch 3 with final weights $W = [-1, 3, -1, 1]$.

Deep Learning Question Bank - Part B (Continued)

Question 3

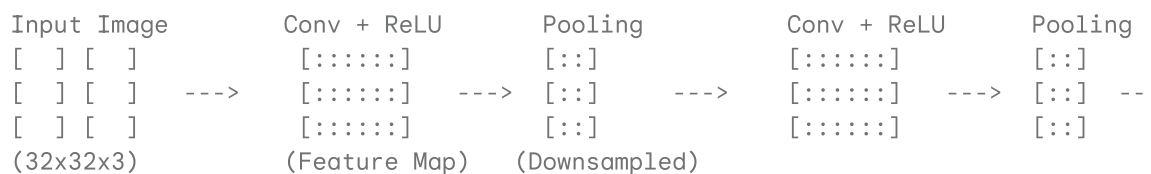
3(a) Briefly discuss the architecture of CNN with a neat diagram. (6 Marks)

Convolutional Neural Network (CNN) architecture is designed to process data with a grid-like topology, such as images. It automatically learns spatial hierarchies of features.

Core Components:

1. **Convolutional Layer (CONV):** The core building block. It uses learnable filters (kernels) that slide (convolve) across the input image to produce **feature maps**. These filters detect patterns like edges, textures, or shapes.
 - *Hyperparameters:* Number of filters, Kernel size (F), Stride (S), Padding (P).
2. **Activation Function (ReLU):** Typically applied after convolution to introduce non-linearity. It replaces negative values with zero ($\max(0, x)$), helping the network learn complex patterns.
3. **Pooling Layer (POOL):** Performs down-sampling to reduce the spatial dimensions (width and height) of the feature map. This reduces the number of parameters and computation, and makes the detection robust to small translations.
 - *Types:* Max Pooling (takes maximum value), Average Pooling.
4. **Fully Connected Layer (FC):** Located at the end of the network. It flattens the output from previous layers into a vector and connects every neuron to every neuron in the next layer, acting like a traditional Multi-Layer Perceptron (MLP) for final classification.

General Architecture Diagram:



3(b) Considering the given matrix, perform the different convolutional operations considering zero padding and stride as 1. (10 Marks)

Given:

- **Input Matrix (I):** 6×6
- **Filter/Kernel (K):** 3×3
- **Stride (S):** 1

- **Padding (P):** Zero Padding (Assuming $P = 1$ to maintain spatial resolution, commonly known as "Same" convolution).

The Kernel:

$$K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

Operation: This filter calculates the difference between the sum of the left column and the sum of the right column of the 3×3 patch covered by the filter. It acts as a **Vertical Edge Detector**.

Calculation Logic (for center pixel $x_{i,j}$):

$$\text{Output}_{i,j} = \text{Sum}(\text{Left Column}) - \text{Sum}(\text{Right Column})$$

Sample Calculations:

1. Top-Left Pixel (0,0): Since we use Zero Padding, the neighbor pixels outside the matrix are 0.

- **Patch:**

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 1 & 5 \end{bmatrix}$$

- Left Column Sum: $0 + 0 + 0 = 0$
- Right Column Sum: $0 + 0 + 5 = 5$
- **Result:** $0 - 5 = -5$

2. Pixel at (1,1) - Original value '5':

- **Patch (from input):**

$$\begin{bmatrix} 3 & 0 & 1 \\ 1 & 5 & 8 \\ 2 & 7 & 2 \end{bmatrix}$$

- Left Column Sum: $3 + 1 + 2 = 6$
- Right Column Sum: $1 + 8 + 2 = 11$
- **Result:** $6 - 11 = -5$

3. Pixel at (1,2) - Original value '8':

- **Patch:**

$$\begin{bmatrix} 0 & 1 & 2 \\ 5 & 8 & 9 \\ 7 & 2 & 5 \end{bmatrix}$$

- Left Column Sum: $0 + 5 + 7 = 12$
- Right Column Sum: $2 + 9 + 5 = 16$
- **Result:** $12 - 16 = -4$

(See the attached Python script for the code to generate the full matrix)

Final Output Matrix (Calculated):

$$\text{Output} = \begin{bmatrix} -5 & -1 & -6 & 0 & 5 & 10 \\ -5 & -5 & -4 & 4 & 4 & 5 \\ -6 & -4 & -11 & 5 & 12 & 3 \\ -5 & -7 & 2 & 1 & 10 & 2 \end{bmatrix}$$

Deep Learning Question Bank - Part B (Continued)

Question 4

4(a) Elaborate how pooling can be used to detect the edges in an image. For the given matrix in Q3b, apply max and average pooling considering the size of 3x3. (6 Marks)

Role of Pooling in Edge Detection: Strictly speaking, **Convolutional Layers** are responsible for *detecting* edges (using filters like Sobel or the one used in Q3b). The **Pooling Layer's** role regarding edges is **Preservation and Robustness**:

1. **Feature Selection:** Max Pooling selects the highest value in a local window. If a strong edge was detected by the convolution layer (resulting in a high activation value), Max Pooling ensures this strong edge information is passed to the next layer, while discarding weaker, noisy signals.
2. **Invariance:** It makes the edge detection robust to small translations or distortions. Even if the edge shifts slightly in the input image, the pooled output remains relatively stable.
3. **Sharpening:** By keeping only the maximum response, Max Pooling tends to sharpen the feature map, highlighting the most prominent edges.

Pooling Calculation:

- **Input:** 6×6 Matrix (from Q3b).
- **Pool Size:** 3×3 .
- **Assumption: Stride = 3** (Non-overlapping). Since the input is 6×6 and pool is 3×3 , a stride of 3 partitions the image perfectly into 4 distinct regions (2×2 output).

Input Matrix:

$$\left[\begin{array}{ccc|ccc} 3 & 0 & 1 & 2 & 7 & 4 \\ 1 & 5 & 8 & 9 & 3 & 1 \\ 2 & 7 & 2 & 5 & 1 & 3 \\ \hline 0 & 1 & 3 & 1 & 7 & 8 \\ 4 & 2 & 1 & 6 & 2 & 8 \\ 2 & 4 & 5 & 2 & 3 & 9 \end{array} \right]$$

1. **Max Pooling (3×3):** Take the maximum value from each block.

- **Top-Left Block:** $\{3, 0, 1, 1, 5, 8, 2, 7, 2\} \rightarrow \text{Max} = 8$
- **Top-Right Block:** $\{2, 7, 4, 9, 3, 1, 5, 1, 3\} \rightarrow \text{Max} = 9$
- **Bottom-Left Block:** $\{0, 1, 3, 4, 2, 1, 2, 4, 5\} \rightarrow \text{Max} = 5$
- **Bottom-Right Block:** $\{1, 7, 8, 6, 2, 8, 2, 3, 9\} \rightarrow \text{Max} = 9$

Max Pooling Output:

$$\begin{bmatrix} 8 & 9 \\ 5 & 9 \end{bmatrix}$$

2. Average Pooling (3×3): Calculate the average of values in each block.

- **Top-Left Block:** Sum = $3 + 0 + 1 + 1 + 5 + 8 + 2 + 7 + 2 = 29$
 - Avg = $29/9 \approx 3.22$
- **Top-Right Block:** Sum = $2 + 7 + 4 + 9 + 3 + 1 + 5 + 1 + 3 = 35$
 - Avg = $35/9 \approx 3.89$
- **Bottom-Left Block:** Sum = $0 + 1 + 3 + 4 + 2 + 1 + 2 + 4 + 5 = 22$
 - Avg = $22/9 \approx 2.44$
- **Bottom-Right Block:** Sum = $1 + 7 + 8 + 6 + 2 + 8 + 2 + 3 + 9 = 46$
 - Avg = $46/9 \approx 5.11$

Average Pooling Output:

$$\begin{bmatrix} 3.22 & 3.89 \\ 2.44 & 5.11 \end{bmatrix}$$

4(b) Assume a series of data stored as x-ray images with reference to classifying the image as presence or absence of chest congestion. The number of images are less for training. Identify and justify the technique use to increase the training set and explain the process in detail. (10 Marks)

Identified Technique: Data Augmentation

Justification:

1. **Dataset Scarcity:** Medical datasets (like X-rays for chest congestion) are often small due to privacy concerns and the cost of expert annotation. Deep learning models require large amounts of data to converge.
2. **Overfitting Prevention:** With a small dataset, a model might memorize specific details (noise) rather than learning general features of congestion. Augmentation artificially creates more samples, forcing the model to learn robust features.
3. **Invariance Learning:** X-rays might be taken at slightly different angles or lighting conditions. Augmentation trains the model to be invariant to these changes (e.g., a slightly rotated lung is still a lung).
4. **Class Imbalance:** If "congestion" cases are rare compared to "normal" cases, augmentation can be applied specifically to the minority class to balance the dataset.

Detailed Process of Data Augmentation: Data augmentation involves creating modified versions of images in the dataset to expand the training size without collecting new clinical data.

1. Geometric Transformations:

- **Rotation:** Rotate the X-ray image by small angles (e.g., $\pm 5^\circ$ to $\pm 10^\circ$). *Note: Large rotations are avoided as X-rays have a fixed upright orientation.*
- **Flipping:** Horizontal flipping (mirroring) is valid for chest X-rays as the lungs generally maintain symmetry relevant to congestion, though the heart position flips (care must be taken if cardiac features are vital, but for congestion, it is usually acceptable).
- **Scaling/Zoom:** Randomly zooming in (e.g., 10-20%) to simulate different distances from the X-ray machine.
- **Translation:** Shifting the image horizontally or vertically by a few pixels.

2. Intensity/Photometric Transformations:

- **Brightness/Contrast Adjustment:** Adjusting the contrast is crucial for X-rays, as bone and tissue density can vary between machines. This helps the model generalize to different X-ray equipment.
- **Gamma Correction:** Non-linear brightness adjustment.

3. Noise Injection:

- Adding small amounts of Gaussian noise or Salt-and-Pepper noise to simulate graininess often found in lower-quality medical scans.

4. Advanced Techniques (Deep Learning based):

- **GANs (Generative Adversarial Networks):** Training a GAN to generate entirely new, synthetic X-ray images that look indistinguishable from real patient data.

Process Workflow:

1. **Input:** Original Dataset (N images).
2. **Pipeline:** Define a set of valid transformations (e.g., Rotate 5° , Flip Horizontal , Increase Contrast).
3. **Generation:** For every image in a batch during training, apply a random combination of these transformations.
4. **Output:** An effectively infinite stream of unique variations of the original X-rays fed into the CNN.

Deep Learning Question Bank - Part B (Continued)

Question 5

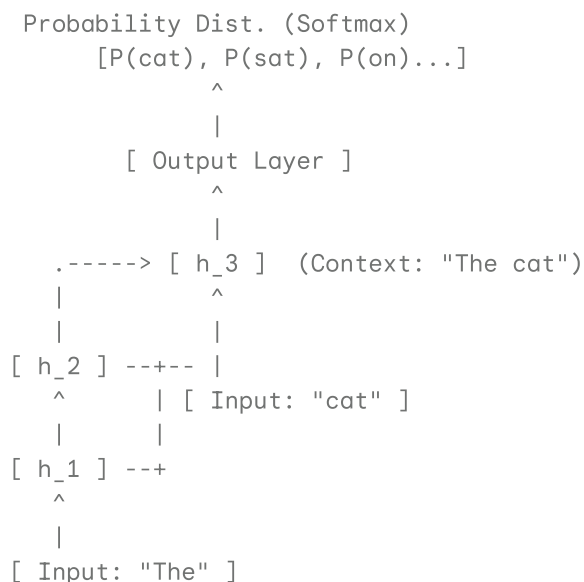
5(a) Illustrate the working of a language model with a neat sketch. (08 Marks)

Definition: A **Language Model (LM)** is a probabilistic model that learns to predict the next word (or character) in a sequence given the context of previous words. Mathematically, it assigns a probability to a sequence of words $P(w_1, w_2, \dots, w_n)$ or estimates the conditional probability of the next word $P(w_t | w_1, \dots, w_{t-1})$.

Working Principle (RNN-based LM):

1. **Input:** A sequence of words (tokens) is converted into embeddings (vectors).
2. **Hidden State:** At each time step t , the network takes the current word embedding (x_t) and the previous hidden state (h_{t-1}) to calculate the new hidden state (h_t). This state represents the "context" or "memory" of the sentence so far.
3. **Output:** The hidden state h_t is passed through a Softmax layer to produce a probability distribution over the entire vocabulary. The word with the highest probability is the predicted next word.

Sketch:



Example:

- **Input Sequence:** "The cat"
- **Target:** "sat"
- The model calculates $P(\text{sat} | \text{The, cat})$ and tries to maximize this probability during training.

5(b) Summarize the difference between Backpropagation Through Time (BPTT) and Bidirectional Recurrent Networks (BiRNN). (08 Marks)

Feature	Backpropagation Through Time (BPTT)	Bidirectional Recurrent Network (BiRNN)
Nature	It is a Training Algorithm (Gradient Descent technique).	It is a Network Architecture (Structure).
Function	Used to calculate gradients and update weights in RNNs by unrolling the network over time.	Used to process input data in two directions simultaneously to capture full context.
Direction of Flow	Calculates errors backward from the last time step to the first ($t = T \rightarrow t = 1$) to adjust weights.	Processes input forward ($t = 1 \rightarrow T$) and backward ($t = T \rightarrow 1$) to generate outputs.
Dependency	Deals with the "Vanishing Gradient" problem during the weight update process.	Deals with the "Future Context" problem (predicting y_t using future inputs x_{t+1}).
Analogy	Like trying to find the cause of an error by tracing back history.	Like reading a sentence fully before deciding the meaning of a specific word.

Question 6

6(a) Illustrate the working of LSTM with appropriate diagram and example. (08 Marks)

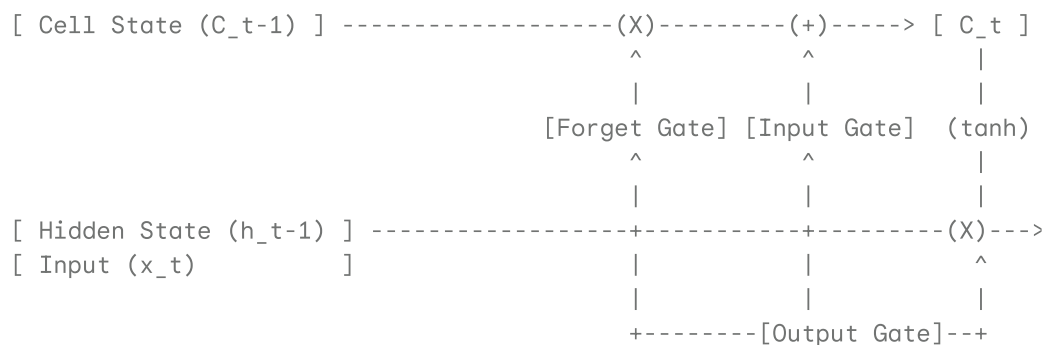
Long Short-Term Memory (LSTM) is a special kind of RNN capable of learning long-term dependencies. It solves the vanishing gradient problem using a mechanism called **Gates**.

Core Concept: The Cell State (C_t) The cell state runs like a conveyor belt through the entire chain, with only minor linear interactions. Gates regulate the flow of information onto or off this state.

The Three Gates:

1. **Forget Gate (f_t):** Decides what information to throw away from the cell state.
 - Sigmoid layer outputs 0 (forget) to 1 (keep).
2. **Input Gate (i_t):** Decides what new information to store in the cell state.
 - Sigmoid layer decides which values to update.
 - Tanh layer creates a vector of new candidate values.
3. **Output Gate (o_t):** Decides what to output based on the cell state.

Diagram:

**Example:**

- **Sentence:** "The clouds are in the *sky*."
- **Context:** When the model sees "clouds", the **Input Gate** adds "clouds" to the cell state. When it sees "sky", it uses the context of "clouds" (stored in Cell State) via the **Output Gate** to predict "sky" instead of "ceiling", even if the words are far apart.

6(b) Justify RNN is more suitable to implement temporal recommender systems when compared to other NN. (08 Marks)**Justification:**

1. **Sequential Nature of Data:** User behavior is inherently sequential. A purchase made today is influenced by purchases made yesterday. RNNs are explicitly designed to handle **sequential data** ($x_1, x_2, \dots, x_t$), unlike standard Neural Networks (MLPs) which treat inputs as independent or static vectors.
 2. **Variable Length History:** Users have history logs of varying lengths (some bought 5 items, some 50). RNNs can process sequences of **arbitrary lengths**, whereas CNNs or MLPs typically require fixed-size inputs.
 3. **Capturing Evolution of Preferences:** User tastes change over time (Temporal Dynamics). An RNN maintains a **Hidden State (Memory)** that evolves with every new interaction. This allows the model to shift its recommendations based on recent trends in the user's session (Short-term interest) while retaining long-term preferences.
 4. **Contextual Ordering:** The **order** matters. Watching Movie A *then* Movie B might imply a different mood than watching B *then* A. RNNs respect this temporal order, whereas Bag-of-Words approaches (often used with MLPs) lose this structural information.
- **Comparison:** A **CNN** might look for local patterns (e.g., did they buy item A and B together?). but it struggles to capture the long-term flow of a user's journey compared to

Deep Learning Question Bank - Part B (Continued)

Question 7

7(a) Briefly discuss the various challenges faced in Reinforcement Learning (RL). (10 Marks)

Reinforcement Learning involves an agent learning to make decisions by interacting with an environment. Unlike supervised learning, it faces several unique and difficult challenges:

1. Exploration vs. Exploitation Dilemma:

- The agent must balance **exploiting** known knowledge to gain high rewards (doing what it knows works) and **exploring** the environment to discover new, potentially better strategies.
- *Challenge:* If it explores too much, it loses rewards. If it exploits too early, it might get stuck in a suboptimal local maximum.

2. Credit Assignment Problem (Delayed Rewards):

- In many scenarios, the reward for an action is not immediate. For example, in Chess, a move made 50 turns ago might be the cause of a victory now.
- *Challenge:* The agent must figure out which specific past action (among thousands) is responsible for the final outcome.

3. Curse of Dimensionality:

- Real-world environments have massive state spaces (e.g., a robot with many joints and sensors).
- *Challenge:* Traditional tabular methods (like Q-tables) fail because the memory required grows exponentially with the number of state variables.

4. Non-Stationarity:

- In supervised learning, the data distribution is fixed. In RL, as the agent learns, its policy changes, which changes the data it observes.
- *Challenge:* The training data distribution is constantly shifting, making the learning process unstable (the "moving target" problem).

5. Partial Observability (POMDP):

- The agent often cannot see the entire state of the environment (e.g., a poker player cannot see opponents' cards).
- *Challenge:* The agent must rely on memory or estimates of the hidden state, adding complexity.

6. Sample Efficiency:

- RL algorithms often require millions of interactions with the environment to learn simple tasks.
- *Challenge:* In real-world robotics, this is slow and can cause wear and tear (or damage) to the hardware.

7(b) With respect to Atari Setting, explain Deep Learning as a function approximator. (6 Marks)

The Problem: In Atari games, the state is the raw screen pixels (210×160 pixels, 3 color channels). The number of possible states is astronomically high ($256^{210 \times 160 \times 3}$). It is impossible to store a Q-value for every state in a table.

The Solution: Function Approximation: Instead of storing values, we use a Deep Neural Network (specifically a Convolutional Neural Network or CNN) to *approximate* the Q-function.

$$Q(s, a; \theta) \approx Q^*(s, a)$$

- **Input:** The raw pixel frames of the game (often preprocessed/stacked).
- **The Function Approximator (CNN):** The network processes the image to extract features (ball location, paddle position).
- **Output:** The network outputs a vector of Q-values, one for each possible action (Up, Down, Left, Right, Fire, etc.).
- **Training:** We adjust the weights θ of the network to minimize the difference between the predicted Q-value and the target Q-value (derived from the Bellman equation).

This allows the agent to generalize: if it learns that "ball near left edge + paddle low" is bad in one situation, it can apply that knowledge to similar-looking pixel states.

Question 8

8(a) Briefly discuss Modeling States Versus State-Action Pairs. (10 Marks)

In Reinforcement Learning, we typically estimate value functions to guide decision-making. There are two primary ways to model these values:

1. Modeling States (State-Value Function, $V(s)$):

- **Definition:** Estimates how good it is for an agent to be in a specific state s .
- **Equation:** $V^\pi(s) = E[R_t | s_t = s]$.
- **Usage:** It tells you "State A is better than State B".
- **Limitation for Control:** Knowing that State A is good doesn't tell you *which action* takes you there. To choose an action, you need a model of the environment (transition probabilities) to look ahead.
 - *Rule:* If you have $V(s)$, you need a Model to act.

2. Modeling State-Action Pairs (Action-Value Function, $Q(s, a)$):

- **Definition:** Estimates how good it is to take a specific action a while in state s .
- **Equation:** $Q^\pi(s, a) = E[R_t | s_t = s, a_t = a]$.
- **Usage:** It tells you "In State A, going Right is worth 10 points, going Left is worth 5 points."
- **Advantage for Control:** This is ideal for **Model-Free RL**. You do not need to know the physics of the world. You simply look at the Q-values for the current state and pick the action with the highest value ($\max_a Q(s, a)$).

Comparison:

- **Complexity:** $Q(s, a)$ is more complex to learn because the space is (States $\times$ Actions), whereas $V(s)$ is just (States).
- **Utility:** $Q(s, a)$ allows for immediate greedy action selection without planning. $V(s)$ is often used in Actor-Critic methods (as the Critic) to help reduce variance.

8(b) With an appropriate example, explain the Straw Man algorithm. (6 Marks)

The "Straw Man" algorithm in the context of Deep Reinforcement Learning refers to a naive approach to applying neural networks to Q-learning, which typically fails due to instability.

The Algorithm (Naive Deep Q-Learning):

1. Let the agent play the game.
2. At every step t , collect data tuple (s_t, a_t, r_t, s_{t+1}) .
3. Immediately perform a standard Supervised Learning update (SGD) on the network using just this one sample.
 - Target $y = r_t + \gamma \max Q(s_{t+1}; \theta)$
 - Update weights to minimize $(y - Q(s_t, a_t; \theta))^2$.

Why it fails (The Example): Imagine a robot learning to navigate a maze.

- **Correlation:** As the robot drives down a long hallway, the input frames s_t, s_{t+1}, s_{t+2} look almost identical.
- **Overfitting:** If the robot drives straight for 100 steps, the "Straw Man" algorithm updates the network 100 times saying "Driving Straight is good".
- **Catastrophic Forgetting:** The network overfits to "always drive straight" and forgets how to turn. When it finally hits a wall (negative reward), it creates a massive gradient that destroys the previous learning.
- *This "Straw Man" motivates the need for **Experience Replay** (breaking correlations by*

Deep Learning Question Bank - Part B (Continued)

Question 9

9(a) Explain the concept of Attention Mechanisms in neural networks. How do Attention Mechanisms enhance performance in tasks such as Machine Translation? (08 Marks)

Concept of Attention Mechanisms: In traditional Sequence-to-Sequence (Seq2Seq) models (Encoder-Decoder), the encoder must compress the entire input sequence into a single fixed-length context vector. This creates a bottleneck, causing information loss for long sequences.

Attention solves this by allowing the decoder to "look back" at the entire input sequence at every step of generation. It assigns a weight (importance score) to each input state, effectively focusing on the relevant parts of the input for the current output.

Core Components:

1. **Query (Q):** What the decoder is currently looking for (current hidden state).
2. **Key (K):** What the encoder has to offer (input hidden states).
3. **Value (V):** The actual information content.
4. **Score:** The match between Query and Key determines the attention weight.

Enhancement in Machine Translation:

- **Alignment:** In translation, words in the source language do not always align one-to-one with the target language (e.g., "La maison rouge" → "The red house"). Attention allows the model to align the word "red" with "rouge" and "house" with "maison" dynamically during generation.
- **Handling Long Sentences:** It removes the reliance on a single context vector, allowing the model to translate long and complex sentences without "forgetting" the beginning of the sentence.
- **Interpretability:** Attention weights can be visualized (heatmaps) to show exactly which source words the model focused on when generating a specific target word.

9(b) Describe the architecture and functioning of a Neural Turing Machine. (08 Marks)

Neural Turing Machine (NTM): An NTM is a neural network architecture coupled with an external memory resource, which it can interact with using attentional mechanisms. It mimics a Turing Machine but is differentiable end-to-end.

Architecture:

1. **Controller:** A neural network (often an RNN or LSTM) that receives inputs and emits outputs. It acts as the CPU.
2. **Memory Bank:** A large matrix ($N \times M$) where information can be stored and retrieved. Unlike LSTM cell states, this memory is explicit and addressable.

3. **Read/Write Heads:** Mechanisms that the controller uses to interact with the memory.

Functioning:

- **Reading:** The controller produces a weight vector (attention) over all memory locations. The Read vector is a weighted sum of the memory rows.

$$r_t \leftarrow \sum_i w_t(i) M_t(i)$$

- **Writing:** Involves two steps:
 - **Erase:** Removes old information based on an erase vector.
 - **Add:** Adds new information based on an add vector.
- **Addressing Mechanisms:** To decide *where* to read/write, NTM uses:
 1. **Content-based Addressing:** Finds memory locations similar to a specific key vector (like a hash map lookup).
 2. **Location-based Addressing:** Moves the head relative to the current position (e.g., "move to the next slot"), enabling sequential operations like copying data.

Question 10

10(a) Discuss the process of training a Generative Adversarial Network (GAN). What are the challenges associated with training GANs, and how can techniques such as Wasserstein GANs help improve training stability? (08 Marks)

Training Process: GAN training involves a **Minimax Game** between two networks:

1. **Generator (G):** Takes random noise (z) and tries to generate fake data ($G(z)$) that looks real.
 2. **Discriminator (D):** Takes both real data (x) and fake data ($G(z)$) and tries to classify them correctly (Real vs. Fake).
- **Objective:** D tries to maximize the accuracy of classification, while G tries to minimize it (fool D).
 - **Cycle:** Train D for k steps $\rightarrow$ Train G for 1 step $\rightarrow$ Repeat.

Challenges:

1. **Mode Collapse:** The generator finds one specific output that fools the discriminator effectively and keeps producing only that single output (ignoring the diversity of real data).
2. **Vanishing Gradients:** If the discriminator is too perfect, the gradients vanish, and the generator learns nothing.
3. **Non-Convergence:** Since they fight against each other, the model often oscillates rather than converging to a stable equilibrium (Nash Equilibrium).

Wasserstein GAN (WGAN):

- **Issue with Standard GAN:** Uses Jensen-Shannon (JS) divergence, which provides no useful gradient if the distributions of real and fake data do not overlap.
- **WGAN Solution:** Uses the **Wasserstein Distance** (Earth Mover's Distance). It measures the "effort" required to transform the fake distribution into the real distribution.
- **Benefit:** Even if the distributions are far apart, WGAN provides a smooth, non-zero gradient, allowing the generator to learn continuously. It significantly improves stability and reduces mode collapse.

10(b) Compare Generative Adversarial Networks (GANs) with Variational Autoencoders (VAEs) in terms of architecture, training process, and applications. (08 Marks)

Feature	Generative Adversarial Networks (GANs)	Variational Autoencoders (VAEs)
Architecture	Two separate networks: A Generator and a Discriminator competing against each other.	Single network structure: An Encoder compresses input to a latent space, and a Decoder reconstructs it.
Training Process	Adversarial (Minimax Game): G tries to fool D ; D tries to catch G . No explicit density estimation.	Probabilistic (ELBO): Maximizes the Evidence Lower Bound. Minimizes reconstruction error + KL Divergence (regularization).
Latent Space	No explicit constraint on the structure of the latent space.	Forces the latent space to follow a Gaussian distribution (smooth and continuous).
Output Quality	Generates sharp, high-detail images , but prone to artifacts and mode collapse.	Generates blurrier images (due to mean squared error loss) but preserves global structure well.
Applications	Image-to-Image translation, Super-resolution, Art generation, Deepfakes.	Anomaly detection, Denoising, Interpolation between features.